



Gila Monster Pulse generator v1.0

To meet the need for a pulse generator capable of triggering external instruments in neurostimulation protocols with multiple frequency patterns, the Gila Monster v1.0 was developed. Although numerous DIY solutions are available in online repositories, this project intentionally adopts a minimalist hardware design, centered on an Arduino Uno built around the ATmega328P microcontroller. The design prioritizes the timing engine at both the hardware and firmware levels to ensure reliable, time-critical pulse generation.

1. Firmware Specifications

- **Stimulation Modes:** Multi-engine pulse generation supporting Continuous (periodic), Burst (trains with inter-burst gaps), and Non-Periodic Stimulation (NPS) with randomized intra-window intervals (see, reference *Cota et al.*)
- **Time Base:** Microsecond-level timing relying on the 16 MHz system clock. Due to AVR hardware prescaling, the time base (*micros()*) advances in 4 μ s increments.
- **Frequency Range & Resolution:** Programmable output frequency from 0.10 Hz to 500.00 Hz, with 0.01 Hz resolution.
- **Pulse Width (PW) Range:** Fixed time from 0.10 ms (Continuous and Burst) or 0.05 ms (NPS) up to 999.99 ms, in 0.01 ms steps, or Duty Cycle in 0.01 % steps. Measured edge accuracy is $\pm 4 \mu$ s in all modes (Section 6.3).
- **Output Control & Switching Latency:** Direct port manipulation using PORTD, bit 7 (PD7), a single 2-cycle instruction (≈ 125 ns at 16 MHz).
- **Output Logic Level:** 5 V TTL-compatible digital output.
- **External Triggering & Synchronization:** Hardware interrupt on Pin D3 (INT1) with user-selectable Rising or Falling edge detection. In Trigger mode, the external trigger then starts the whole programmed session, exactly as switching State ON does in Manual mode, and the session Timer is counted from the trigger. Triggers arriving while a session is running are ignored. The interrupt registers the event and the session starts in the main loop, with a measured trigger-to-output latency of ~ 30 to $\sim 50 \mu$ s.
- **Single-Shot Diagnostic:** Manual pulse via Pin D6, debounced in firmware (100 ms of stable contact), delivering one pulse with the programmed width per press. Interlocked with the master state: operational only when State is OFF.

- **Duty Cycle Safety Clamp:** The output always returns LOW for at least 100 μ s between pulses, which prevents 100 % duty cycle (DC latching). The maximum pulse width is $PW_{max} = T - 100 \mu s$, applied to the displayed value.
- **Parameter Storage:** Non-volatile EEPROM storage of the 13 operational parameters, protected by a signature. Absent or invalid data loads the default settings.
- **Serial Communication:** UART interface (9600 baud) reporting the active configuration, the effective pulse timing in microseconds, a report of the last session and NPS simulation data for offline analysis.

2. Operating Engines

2.1 Deterministic Engine (Continuous [Periodic] / Burst Modes)

The deterministic timing engine computes the output period (T) as a direct function of the programmed frequency (f), according to:

$$T (\mu s) = 1,000,000 / f (\text{Hz})$$

Each rising edge is scheduled at the previous scheduled edge plus T, and each falling edge at the rising edge plus PW. Because edges are referenced to this absolute grid, the long-term frequency is exact and the per-pulse error does not accumulate.

In **Continuous Mode**, pulses are generated at a fixed period until the Timer expires or the user stops.

In **Burst Mode**, pulse delivery is governed by:

- a pulse counter defining the exact number of pulses per train (Count), delivered at period T;
- a silent-state timer (Gap), defined as the silence from the falling edge of the last pulse of a train to the rising edge of the first pulse of the next train. The resulting cycle time is:

$$\text{Cycle} = (\text{Count} - 1) \times T + PW + \text{Gap}$$

A Gap of 0 ms produces a single train, after which the generator switches itself OFF.

2.2 Non-Periodic Stimulation (NPS) Engine

The Non-Periodic Stimulation (NPS) engine implements a constrained uniform distribution algorithm to generate temporally irregular pulse trains while preserving overall pulse density. Although the underlying random selection is uniform within constrained temporal bounds, the sequential dependency between events produces a heavy-tailed distribution of inter-stimulus intervals (ISI). As a result, shorter ISIs occur more frequently while longer intervals remain possible, yielding a power-law-like temporal structure rather than a purely Poisson or periodic pattern.

- **Algorithm** (1,000 ms window, $t_0 = 0$):

$$t_k = \max(\text{Random}(t_{k-1}, \text{End}_k), t_{k-1} + \text{IT}_{\min})$$
$$\text{End}_k = 1000 - (N - k) \times \text{IT}_{\min}$$

where:

N	number of pulses in the window (Pulse Count)
k	index of the pulse being placed, from 1 to N
t_k	timestamp of pulse k , in ms from the start of the window ($t_0 = 0$)
$\text{IT}_{\min}$	minimum interval between two pulses, in ms
End_k	latest time pulse k can occur while still leaving $\text{IT}_{\min}$ for each of the remaining $N - k$ pulses
<i>Random</i>	uniform draw in milliseconds

Each timestamp is drawn uniformly between the previous one and the remaining window space, and then pushed forward if it falls closer than $\text{IT}_{\min}$.

- **Window Size:** Fixed 1,000 ms temporal window. Timestamps are integer milliseconds.
- **Pulse Density:** Exactly N pulses per window (Pulse Density Modulation). With $N \times \text{IT}_{\min} < 1000$ ms every timestamp falls inside the window, and every interval, including across consecutive windows, is at least $\text{IT}_{\min}$.
- **Temporal Structure:** Inter-stimulus intervals (ISI) follow an inverse distribution of the form $f(x) = k/x$, that is, a power law with unitary exponent ($\beta = 1$). With the parameters of the original reference, $N = 4$ and $\text{IT}_{\min} = 20$ ms, intervals range from 20 ms to 1860 ms. Other combinations of N and $\text{IT}_{\min}$ change the shape of the interval distribution and should be verified with the Comm export before use.

References by Cota, et al.

<https://doi.org/10.1016/j.yebeh.2019.106609>

<https://doi.org/10.1016/j.yebeh.2008.09.006>

3. Parameter Matrix & Data Mapping

ID	Parameter	Type	Logic / Range	Resolution
00	Mode	Option	Manual (0) / Triggered (1)	N/A
01	Trigger Edge	Option	Falling (0) / Rising (1) (<i>Triggered mode only</i>)	N/A
02	Type	Option	Continuous (0) / Burst (1) / NPS (2)	N/A
03	State	Action	OFF / ON	N/A
04	Frequency	Decimal	0.10 – 500.00 Hz (<i>Cont. and Burst mode only</i>)	0.01 Hz
05	Pulse Count	Integer	1 – 999 pulses (<i>Burst mode</i>) / 1 – 50 pulses and N × ITImin < 1000 ms (<i>NPS mode</i>)	1 pulse
06	Gap	Integer	0 – 999,999 ms (<i>Burst mode only; 0 = single train</i>)	1 ms
07	Minimum ITI	Integer	1 – 999 ms (<i>NPS mode only</i>)	1 ms
08	Width	Option	Fixed (0) / Duty Cycle (1) (<i>NPS: Fixed only</i>)	N/A
09	Pulse	Decimal	Fixed: 0.10 – 999.99 ms (NPS: from 0.05 ms), limited to T – 0.1 ms (NPS: ITImin – 0.1 ms). Duty Cycle: equivalent limits in %	0.01 ms / 0.01 %
10	Timer	Integer	0 (Disabled) – 999,999 ms	1 ms
11	Save	Action	“SAVED”	N/A
12	Comm	Action	“SENT”	N/A

Several limits depend on other parameters (Frequency, Minimum ITI and Pulse Count). When a change makes another value invalid, that value is corrected and the LCD shows **Auto-adjusted** followed by the name of the parameter that changed. Parameters that do not apply to the selected mode are hidden from the menu.

4. Hardware Interface (Pinout)

Digital & Analog Pin Assignment

MCU	Arduino				
Pin	Pin	Direction	Pull-Up	Function	Notes
PD7	D7	Output	Disabled	Pulse Output	High-speed TTL output. i.e. use 220 Ω series resistor for opto-isolation or long cables.
PD3	D3	Input	Enabled	Hardware Trigger (INT1)	External interrupt input, Rising or Falling edge. The pull-up keeps the line HIGH when unconnected.
PD2	D2	Input	Enabled	Encoder Channel A (CLK)	Quadrature decoding. Internal pull-up reduces noise and wiring complexity.
PD5	D5	Input	Enabled	Encoder Channel B (DT)	Quadrature decoding. Matched with Channel A.
PD4	D4	Input	Enabled	Encoder Push Button (SW)	Active-low user input. Debounced in firmware.
PD6	D6	Input	Enabled	Single Shot Button	Manual pulse fire. Debounced (100 ms) and interlocked with master state (State OFF only).
PC4	A4	I/O	Enabled	I ² C SDA	Shared data line. Standard-mode 100 kHz . External pull-ups optional if LCD provides them.
PC5	A5	Output	Enabled	I ² C SCL	Shared clock line. Standard-mode 100 kHz .

5. Serial Communication & Data Validation

The COMM [12] function operates as a diagnostic and validation interface, providing access to internal configuration and timing behavior via the serial port.

- **Protocol:** UART serial communication at 9600 baud, 8-N-1.
- **Configuration:** mode, output type and the parameters of the active type, including the effective period and the effective HIGH time of the pulse in microseconds.
- **Last Session Report:** number of pulses fired and number of timing faults (edges placed more than 50 μ s late). A correct session reports 0 timing faults.
- **Monte Carlo Validation (NPS Mode):** the firmware simulates an NPS session with the current parameters and exports the resulting inter-stimulus intervals as a single CSV column [ITI_ms], for offline analysis of ISI variability and power-law fitting. The simulated duration is the programmed Timer, or 10 s when the Timer is disabled.

Notes for open-hardware users:

- The simulation uses the same generator as the stimulation engine, so the exported intervals are those the device would deliver.
- COMM and SAVE are only available when State is OFF. Simulation and data export never run concurrently with stimulation.

6. Operational Constraints and Architectural Limitations

6.1 Firmware Logic & Safety Constraints

- **Parameter Limits:** Limits are applied to the parameter values. When a change forces another parameter to change, the LCD shows Auto-adjusted and the name of that parameter.
- **Pulse Width Limits:** Minimum HIGH time of 100 μ s in Continuous and Burst modes and 50 μ s in NPS mode. Minimum LOW time of 100 μ s between pulses, so $PW_{max} = T - 100 \mu s$ (Continuous/Burst) or $ITI_{min} - 100 \mu s$ (NPS), applied to the displayed value. The firmware cannot produce a constant-high output: every pulse is followed by at least 100 μ s at logic low, which prevents continuous current injection into the preparation.
- **NPS Window Constraint:** The pulses of a window must fit inside it: $N \times ITI_{min} < 1000$ ms. The menu enforces this by limiting the Pulse Count to the largest value that satisfies it, so increasing the Minimum ITI may reduce the Pulse Count, which is signaled by Auto-adjusted.

- **NPS Pulse Width:** In NPS mode the pulse width is always a fixed time, since there is no period to refer a duty cycle to.
- **NPS Pulse Count Hard Limit:** The maximum number of pulses per window is 50. Timestamps are held in two statically allocated buffers of 50 entries (the window being played and the next window being prepared) in the ATmega328P's 2 KB SRAM. Increasing this value would risk memory exhaustion and undefined behavior.
- **Locked Interface:** While State is ON, the knob is ignored, the LCD is not redrawn and Save/Comm are unavailable. The only accepted input is a click on State, which stops the generator.
- **Clean Stop:** Stopping, by click or by the Timer, always lets the current pulse finish. A pulse is never truncated or prolonged by the stop command. A stop during a Burst ends the train after the current pulse.
- **Session Timer:** The session starts when State is switched ON in Manual mode, or at the trigger in Trigger mode. Only pulses whose rising edge is scheduled before the end of the session are emitted. At 1 Hz with Timer = 10,000 ms, exactly 10 pulses are delivered (at 0, 1, ..., 9 s). In Burst mode, the last train may be cut by the end of the session.
- **Trigger Operation:** In Trigger mode, switching State ON arms the generator without producing any output. The external trigger then starts the whole programmed session, a continuous train, a sequence of bursts or a sequence of NPS windows, exactly as switching State ON does in Manual mode. Triggers arriving while a session is running are ignored, so a session always runs as programmed. When the session ends, the generator is armed again and the next trigger starts a new one. With Timer = 0 a session has no end and runs until stopped, so only one trigger is accepted.
- **Single-Shot Diagnostic Logic:** The Single-Shot button (D6) is only operational when State is OFF. A press is accepted after 100 ms of stable contact and delivers one pulse with the programmed width, regardless of how long the button is held. Presses shorter than 100 ms are ignored, which limits manual firing to about 5 presses per second.

6.2 Timing Architecture & Hardware-Induced Limitations

- **Polling Engine with Scheduled Edges:** The main loop compares *micros()* with the next scheduled edge. When an edge is less than 400 μ s away, the CPU busy-waits for it with interrupts masked and then writes the port. No interrupt can run between the time check and the port write.

- **Masked Interval:** Interrupts are masked for at most about 0.8 ms per edge, less than the 1.024 ms Timer0 overflow, so the system clock never loses counts. Encoder and trigger interrupts that occur during this interval are deferred.
- **Resolution Limit:** The time base advances in 4 μs steps, which bounds the placement error of each edge to about $\pm 4 \mu\text{s}$. Sub-microsecond resolution would require dedicated hardware timers (e.g. CTC mode), which this architecture does not use.
- **Clock Tolerance:** The ATmega328P on the Arduino Uno R3 is clocked by a ceramic resonator, typically accurate to about $\pm 0.5 \%$. All timings scale with it (e.g. a programmed 500 Hz may measure between about 497.5 and 502.5 Hz). Relative timing within a protocol is unaffected. Measuring the actual frequency of each unit once with an oscilloscope or frequency counter is recommended.
- **Trigger Latency:** The trigger interrupt only registers the event; the session starts when the main loop processes it. The resulting latency (~ 30 to $\sim 50 \mu\text{s}$) varies from trigger to trigger. When sub-100 μs synchronization matters, record the D7 output together with the external signal and use D7 as the timing reference.
- **Timing Fault Monitor:** Any edge placed more than 50 μs late is counted and reported by Comm, providing a per-session integrity check.

6.3 Temporal Accuracy

Pulse width and period:

Configuration	Pulses	Programmed PW	Measured PW	Programmed T	Measured T
Cont. 1 Hz	60	100 μs	97.0 – 103.3 μs	1,000,000 μs	$\pm 3.6 \mu\text{s}$
Cont. 20 Hz	400	5 ms	4,997.0 – 5,004.0 μs	50,000 μs	$\pm 3.6 \mu\text{s}$
Cont. 500 Hz	9,999	100 μs	97.0 – 103.8 μs	2,000 μs	$\pm 3.8 \mu\text{s}$
Burst 50 Hz $\times$ 10	100	2 ms	1,996.9 – 2,003.4 μs	20,000 μs	$\pm 3.4 \mu\text{s}$
NPS 4/s	120	100 μs	97.1 – 103.5 μs	–	–
NPS 50/s	999	50 μs	49.2 – 54.7 μs	–	–

100 % of the measured pulse widths and periods fell within $\pm 4 \mu\text{s}$ of the programmed value (99.7 % for the 50 μs NPS pulses). The mean frequency at 500 Hz was 499.9999 Hz.

Protocol structure and counting:

Test	Expected	Measured
Burst: pulses per train (50 Hz, Count 10)	10	10 in 10/10 trains
Burst: Gap (Gap = 1,818 ms)	1,818,000 μ s	1,817,997.8 – 1,818,000.3 μ s
Burst: cycle time	2,000,000 μ s	1,999,997.8 – 2,000,002.3 μ s
NPS: pulses per window (N = 4, ITImin 20 ms)	4	4 in 28/28 windows
NPS: pulses per window (N = 50, ITImin 19 ms)	50	50 in 18/18 windows
NPS: shortest interval (ITImin 20 ms / 19 ms)	$\geq$ ITImin	19.998 ms / 18.996 ms
Timer: 1 Hz, Timer = 10,000 ms	10 pulses	10 pulses
Trigger: session start (1 Hz, Timer = 10,000 ms)	10 pulses	10 in every complete session
Trigger: triggers arriving during a session	ignored	6 of 12 ignored
Trigger: trigger-to-output latency (114 triggers)	–	28.9 – 51.3 μ s (mean 37.8, p99 46.7)

The NPS shortest intervals are below ITImin by at most 4 μ s, the resolution of the time base.

7. Operation Modes

7.1 Continuous Mode (Periodic)

Generates a constant flow of pulses at a fixed frequency. This is the simplest operational mode, suitable for protocols requiring regular, periodic stimulation without temporal structure.

Example Configuration: 20 Hz stimulation with 5 ms pulse width.

Parameter	Value
Type	Cont.
Freq	020.00 Hz
Width	Fixed
Pulse	005.00 ms
State	ON

Measured: pulse width 4,997.0 – 5,004.0 μ s; period 50,000 $\pm$ 3.6 μ s (duty cycle 10 %).

7.2 Burst Mode

Generates groups of pulses (trains) separated by a configurable silence interval (Gap). Suitable for protocols requiring episodic stimulation with controlled inter-train intervals.

Example Configuration: 10 pulses at 50 Hz with 2 ms pulse width, repeating every 2 seconds.

Parameter	Value
Type	Burst
Freq	050.00 Hz
Count	010
Gap	001818 ms
Width	Fixed
Pulse	002.00 ms
State	ON

Note: Cycle = (Count – 1) × T + PW + Gap. At 50 Hz, 10 pulses span 9 × 20 ms + 2 ms = 182 ms. For a 2,000 ms cycle, Gap = 2,000 – 182 = 1,818 ms.

Measured: 10 pulses in every train; cycle 2,000,000 ± 2.3 μs; pulse width 1,996.9 – 2,003.4 μs.

7.3 NPS Mode (Non-Periodic Stimulation)

Generates a specific number of pulses per 1,000 ms window at randomized intervals. Suitable for protocols requiring temporal unpredictability while preserving overall pulse density.

Example Configuration: 4 random pulses per second with 100 μ s pulse width, at least 20 ms between pulses.

Parameter	Value
Type	NPS
Count	004
ITlmin	020 ms
Width	Fixed
Pulse	000.10 ms
State	ON

NPS Constraint Validation:

$N \times \text{ITlmin} = 4 \times 20 \text{ ms} = 80 \text{ ms} < 1,000 \text{ ms}$

Pulse count within hard limit: $4 \leq 50$

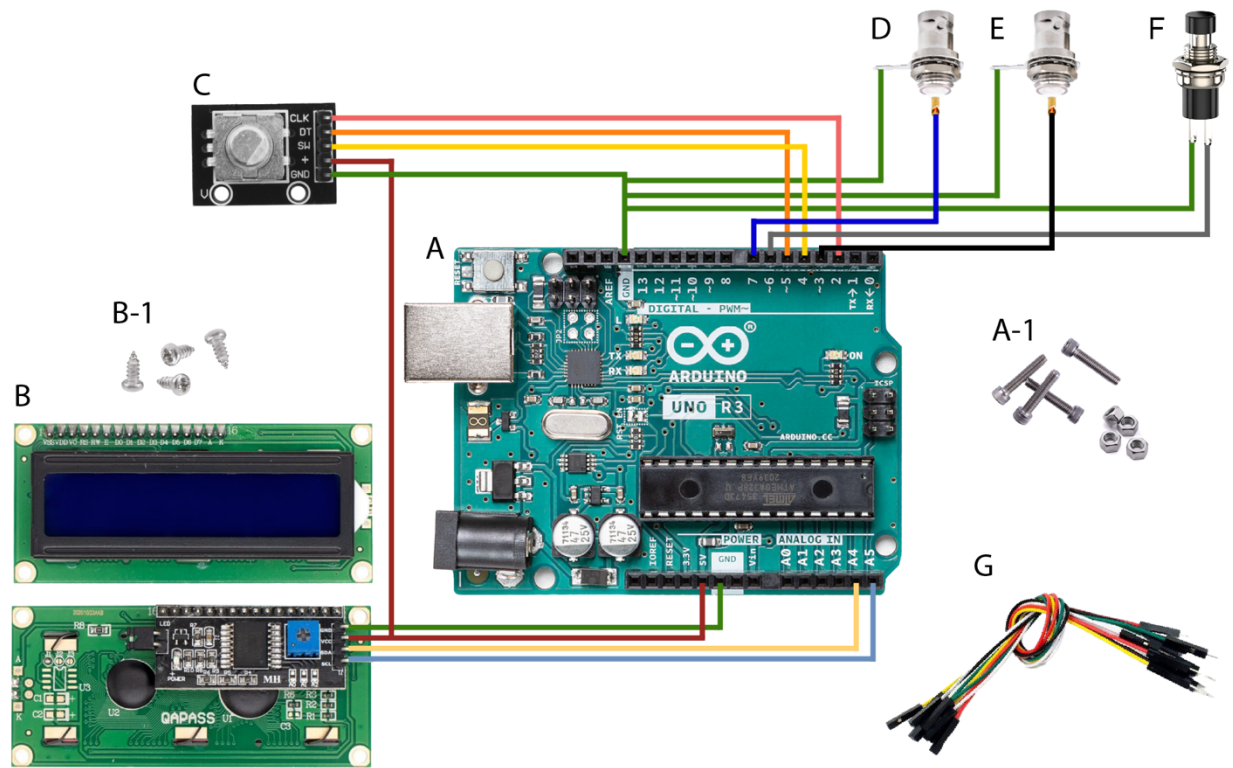
Pulse width within limit: $0.10 \text{ ms} \leq \text{ITlmin} - 0.1 \text{ ms} = 19.9 \text{ ms}$

Measured: 4 pulses in every window; shortest interval 19.998 ms; pulse width 97.1 – 103.5 μ s.

Recommended: before an experiment, use Comm to export the simulated ISI sequence for the chosen Count and Minimum ITI and check the interval distribution offline.

8. Assembly Instructions

8.1 Electronics



	Materials	Quantity	Link	\$
A	Arduino Uno R3	1	Mouser electronics	25.87
A-1	M2 x 12 mm Allen Bolt Socket Cap Screws M2 Hex Nut for Arduino	4	-	-
B	LCD module 16x2, (WxHxD): 80x36x13 mm I2C LCD display module	1	Amazon	10
B-1	Round Head Tap Screw for LCD Ø 2-2.5 mm, L = 3-6 mm)	4	-	-
C	360 Rotatory encoder module KY-040	1	Amazon	~7 (2 un)
D, E	Female BNC panel-mount	2	-	-
F	Push Button RS PBS 110	1	Amazon	~.60
G	Cables, different colors (e.g. Ø 2.5 mm, cross section 0,5 mm², length 10-15 cm)	~15	Amazon	~5 (40 un)

8.2 3D parts

- PLA case (WxHxD): 7.5 x 6 x 8.5 cm
- 4 M2 x 20 mm Allen Bolt Socket Cap Screws

